

Содержание

Введение.....	5
1 Исследовательский раздел	7
1.1 Актуальность выбранной темы	8
1.2 Цель и задачи работы.....	9
1.3 Анализ предметной области	11
1.4 Анализ существующих аналогов	12
1.5 Выбор и описание технологий	15
1.5.1 Язык программирования C#	15
1.5.2 Платформа Windows Forms (WinForms)	17
1.5.3 Формат хранения данных JSON	17
1.5.4 Среда разработки Visual Studio	18
2 Специальный раздел	20
2.1 Подходы к проектированию программного продукта	20
2.2 Проектирование модели данных	21
2.3 Структура и содержание хранилища данных	22
2.4 Логическая схема работы приложения	23
3 Технологический раздел.....	25
3.1 Разработка пользовательского интерфейса	25
3.2 Программная реализация модели данных	27
3.3 Реализация логики хранения (JSON)	27
3.4 Обработка событий и валидация.....	28
3.5 Тестирование работоспособности.....	29
Заключение	31
Список использованных источников.....	33

					УП.340000.000			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.		Шемениев В.В			Отчет по курсовой работы		Лист	Листов
Пров.		Запорожец О.И.					4	34
Реценз.						ТИ (филиал) ДГТУ в г. Азове		
Н. Контр.								
Утв.		Чумак И.В.						

Введение

В современном мире здоровый образ жизни и регулярные физические нагрузки стали неотъемлемой частью жизни активного человека. Систематический подход к тренировкам требует не только дисциплины, но и качественного мониторинга прогресса. Традиционные методы ведения записей, такие как бумажные дневники, постепенно уступают место цифровым решениям. Однако многие существующие мобильные и веб-приложения перенасыщены рекламой, требуют постоянного интернет-соединения или платных подписок. Разработка локального программного продукта на языке C# позволяет создать персонализированный, конфиденциальный и быстрый инструмент для планирования тренировочного процесса, который будет работать полностью автономно.

Объектом исследования является процесс организации и планирования спортивных тренировок пользователя.

Предметом исследования выступают методы и программные средства автоматизации учета физических нагрузок с использованием объектно-ориентированного программирования.

Целью работы является проектирование и разработка функционального приложения «Планировщик тренировок» с интуитивно понятным графическим интерфейсом и системой сохранения данных для обеспечения непрерывного контроля спортивных достижений.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить специфику предметной области и определить требования к функционалу программы.
2. провести сравнительный анализ существующих аналогов и обосновать выбор технологий разработки (C#, .NET, JSON).
3. спроектировать архитектуру приложения и структуру хранения данных.

					УП.340000.000	Лист.
						5
Изм.	Лист	№ докум.	Подп.	Дата		

4. разработать пользовательский интерфейс, обеспечивающий удобное взаимодействие с системой.
5. реализовать программную логику добавления, отображения и сохранения информации.
6. провести тестирование программного продукта на предмет корректности работы и обработки ошибок ввода.

Научная и практическая значимость работы заключается в создании прикладного инструмента, который может быть использован как начинающими атлетами, так и опытными спортсменами для структурирования своих занятий. Проект также демонстрирует применение современных методов сериализации данных для обеспечения долгосрочного хранения информации без использования громоздких систем управления базами данных.

Курсовой проект состоит из введения, трех разделов (исследовательского, специального и технологического), заключения, списка использованных источников и приложений с листингом кода.

					УП.340000.000	Лист.
						6
Изм.	Лист	№ докум.	Подп.	Дата		

1 Исследовательский раздел

Исследовательский раздел данной работы посвящен глубокому анализу предметной области и изучению теоретических основ создания систем управления персональными данными на примере разработки приложения «Планировщик спортивных тренировок». В эпоху повсеместной цифровизации и роста интереса к здоровому образу жизни концепция автоматизации учета физических нагрузок приобретает особую значимость, превращая рутинный процесс ведения бумажных дневников в структурированный цифровой мониторинг прогресса.

В основе проектируемой программы лежит классическая механика логгирования тренировочной активности, где пользователю необходимо фиксировать параметры выполненных упражнений, такие как количество подходов и повторений, с привязкой к конкретным временным интервалам. Исследование начинается с детального изучения методологии построения тренировочных планов, так как успех подобного приложения напрямую зависит от точности отображения структуры занятия и возможности удобного ретроспективного анализа данных.

Важной частью раздела является системный анализ существующих программных решений — от громоздких мобильных платформ до универсальных табличных редакторов. Это позволяет выделить наиболее эффективные подходы к организации интерфейса ввода данных и выявить пробелы в функциональности (например, избыточную сложность или зависимость от интернет-соединения), которые устраняются в рамках данной курсовой работы за счет создания легковесного и автономного приложения.

Техническая составляющая исследования охватывает вопросы проектирования и обработки структур данных. Проектирование качественного планировщика требует не просто наличия полей для ввода, но и разработки архитектуры классов, способной эффективно описывать сущность

					УП.340000.000	Лист.
Изм.	Лист	№ докум.	Подп.	Дата		7

«Тренировка», обеспечивая целостность информации при сохранении и загрузке.

Особое внимание уделяется анализу технологий сериализации. В данном разделе обосновывается использование формата JSON как оптимального средства для хранения локальной базы данных. Данный формат обеспечивает высокую скорость чтения-записи и сохраняет структуру объектов без необходимости использования сложных систем управления базами данных. Кроме того, рассматриваются требования к проектированию пользовательского интерфейса на платформе Windows Forms. Изучение предметной области подтверждает необходимость использования специализированных элементов управления для минимизации ошибок ввода и обеспечения высокой отзывчивости программы.

Таким образом, исследовательский этап закладывает комплексный теоретический фундамент, объединяющий спортивную методологию, принципы объектно-ориентированного проектирования и современные стандарты разработки программного обеспечения на языке C#. Это позволяет перейти к непосредственному проектированию архитектуры и реализации функционала приложения.

1.1 Актуальность выбранной темы

В современном обществе наблюдается устойчивый тренд на поддержание здорового образа жизни и систематические занятия спортом. Ключевым фактором прогресса в тренировках является дисциплина и ведение учета нагрузок.

Многие пользователи сталкиваются с проблемой выбора инструмента для фиксации тренировок:

- Бумажные носители неудобны в хранении, не позволяют быстро анализировать прогресс и подвержены физическому износу.

					УП.340000.000	Лист.
Изм.	Лист	№ докум.	Подп.	Дата		8

- мобильные приложения часто перегружены рекламой, требуют платных подписок для доступа к истории или зависят от наличия интернет-соединения.

- облачные таблицы сложны в использовании на ходу и не имеют специализированного интерфейса.

Разработка локального приложения на языке C# позволяет создать легковесный, бесплатный и полностью автономный инструмент, который обеспечивает быстрый ввод данных и надежное хранение информации на персональном компьютере пользователя.

1.2 Цель и задачи работы

Целью данного курсового проекта является проектирование и программная реализация автоматизированного рабочего места (АРМ) для планирования и учета спортивных тренировок. Разрабатываемое программное обеспечение призвано упростить процесс мониторинга физических нагрузок, обеспечивая пользователю удобный инструмент для хранения и анализа персональных спортивных достижений в единой цифровой среде.

Для достижения поставленной цели в рамках работы необходимо решить следующий комплекс задач:

1. Системный анализ предметной области: Изучить специфику и структуру данных, необходимых для полноценного описания тренировочного процесса, включая качественные и количественные характеристики упражнений.

2. Проведение сравнительного анализа: Исследовать существующие программные аналоги на рынке информационных технологий, выявить их преимущества и недостатки для обоснования уникальности разрабатываемого продукта.

3. Технологическое обоснование: Провести критический отбор и обосновать выбор языка программирования C#, платформы .NET и среды разработки Visual Studio как наиболее эффективного стека для решения поставленных задач.

4. Проектирование архитектуры и данных: Разработать логическую структуру программного продукта, включая проектирование объектной модели данных и выбор формата персистентного хранения информации (JSON).

5. Программная реализация: Разработать интуитивно понятный графический интерфейс пользователя (GUI) и реализовать программную логику обработки событий, обеспечивающую стабильное функционирование системы.

6. Верификация и тестирование: Провести функциональное тестирование готового приложения для подтверждения корректности алгоритмов сохранения, загрузки и отображения данных о тренировках.

Объектом исследования является процесс автоматизации учета спортивных нагрузок. Предметом исследования выступают методы и средства разработки прикладного программного обеспечения на языке C# с использованием технологий событийного программирования и сериализации данных.

1.3 Анализ предметной области

Предметной областью данного исследования является процесс методической организации и систематизации силовых и кардио-тренировок. В современном контексте физической культуры эффективное управление тренировочным процессом невозможно без качественного мониторинга показателей, что ставит задачу перевода физических активностей в плоскость цифровых данных. С точки зрения системного анализа и автоматизации, тренировочный план рассматривается как сложная динамическая структура, которую можно декомпозировать на ряд ключевых взаимосвязанных сущностей:

- Упражнение: представляет собой первичную информационную единицу, определяющую характер физического воздействия. Название упражнения служит идентификатором, позволяющим классифицировать активность по группам мышц или типам нагрузки.
- Нагрузка: выражается через совокупность количественных показателей, таких как объем выполненной работы (количество подходов) и интенсивность (количество повторений в каждом цикле). Эти параметры являются базисом для расчета прогрессии нагрузок, необходимой для достижения спортивных результатов.
- Периодичность: обеспечивает временную детерминацию активности. Привязка каждой тренировки к конкретной дате в календаре позволяет формировать хронологическую последовательность событий, что критически важно для отслеживания регулярности занятий и восстановления организма.

Процесс функционирования разрабатываемой информационной системы строится на строгом соблюдении жизненного цикла данных, который включает в себя три фундаментальных этапа:

					УП.340000.000	Лист.
Изм.	Лист	№ докум.	Подп.	Дата		11

1. **Интерактивный ввод данных:** процесс преобразования физических параметров выполненной работы в программные объекты. На этом этапе формируется первичная запись, проходящая первичную фильтрацию и валидацию для исключения смысловых ошибок.
2. **Персистентное хранение:** важнейший этап, обеспечивающий сохранность информации в энергонезависимой памяти. Запись данных во внешний файл гарантирует возможность доступа к истории тренировок после завершения сеанса работы программы или перезагрузки вычислительной системы, что превращает разрозненные записи в полноценную базу знаний.
3. **Структурированный просмотр:** визуализация накопленного опыта. Вывод истории тренировок в структурированном виде позволяет пользователю проводить ретроспективный анализ своей деятельности, оценивать динамику показателей и вносить коррективы в будущие планы на основе объективных данных, а не субъективных ощущений.

Таким образом, детальный анализ предметной области позволяет сформировать требования к архитектуре приложения, гарантируя, что программный продукт будет не просто хранилищем текста, а эффективным инструментом управления спортивным прогрессом.

1.4 Анализ существующих аналогов

Для выбора оптимальной стратегии разработки был проведен анализ наиболее популярных решений для учета тренировок в виде таблицы (таблица 1.1). Аналоги важный элемент в разработке ПО, многие интересные элементы можно найти в них.

В процессе исследования предметной области был проведен критический анализ существующих программных решений для учета физических нагрузок. Это необходимо для выявления наиболее эффективных механик

					УП.340000.000	Лист.
						12
Изм.	Лист	№ докум.	Подп.	Дата		

взаимодействия с пользователем и определения функциональных пробелов, которые должна устранить разрабатываемая программа.

Таблица 1.1 - Аналоги

Критерий	Бумажный дневник (рис. 1.1)	Мобильное приложение (рис. 1.2)	Таблицы (рис. 1.3)	Разрабатываемое ПО (рис. 1.4)
Удобство ввода	Высокое	Среднее	Низкое	Высокое
Стоимость	Цена тетради	Подписка/Реклама	Бесплатно	Бесплатно
Автономность	Полная	Зависит от облака	Полная	Полная
Анализ данных	Вручную	Автоматически	Требует формул	Автоматически
Приватность	Высокая	Низкая	Высокая	Высокая



Рисунок 1.1 - Бумажный дневник



Рисунок 1.2 – Пример приложения

Рисунок 1.3 – Пример таблицы

Рисунок 1.4 – Пример интерфейса ПО

Вывод: Существующие решения либо слишком сложны, либо навязывают облачные сервисы и подписки. Разрабатываемое приложение займет нишу простых, бесплатных настольных инструментов для персонального использования.

1.5 Выбор и описание технологий

Раздел «Выбор и описание технологий» предназначен для детального обоснования программного стека, использованного при создании приложения. Выбор инструментов не является произвольным; он продиктован требованиями к функциональности, производительности и удобству поддержки программного продукта.

Вот технологии которые были использованы в создании приложения для курсовой работы:

1.5.1 Язык программирования C#

C# (рис. 1.5) - это современный объектно-ориентированный язык программирования, разработанный корпорацией Microsoft как основной язык для платформы .NET. Он сочетает в себе высокую производительность,

характерную для C++, с безопасностью и простотой разработки, присущей Java.

Ключевые преимущества для проекта:

1. Строгая типизация: Предотвращает ошибки, связанные с неверным использованием типов данных на этапе написания кода.
2. автоматическое управление памятью: Благодаря системе сборки мусора разработчику не нужно вручную удалять объекты из памяти, что снижает риск утечек и зависаний программы.
3. богатая стандартная библиотека: Содержит готовые инструменты для работы со списками, датами и потоками ввода-вывода, что критически важно для обработки данных о тренировках.
4. поддержка LINQ: Позволяет писать лаконичные запросы к коллекциям данных (например, для фильтрации тренировок по дате или названию).

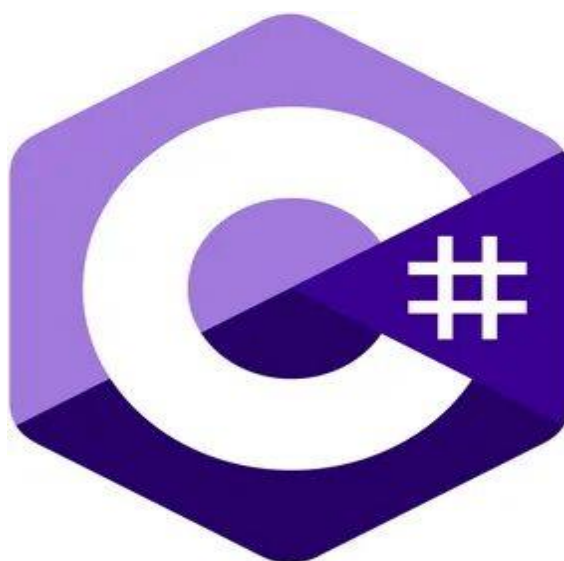


Рисунок 1.5 – C#

1.5.2 Платформа Windows Forms (WinForms)

Windows Forms (рис. 1.6) — это классический графический программный интерфейс (API) для разработки настольных приложений под ОС Windows. В основе технологии лежит событийная модель программирования: программа ожидает действий пользователя (клик, ввод текста) и реагирует на них через обработчики событий.

Особенности применения:

1. **Визуальный дизайнер:** Позволяет проектировать интерфейс методом "Drag-and-Drop" (перетаскивание элементов управления на форму), что значительно ускоряет прототипирование.
2. **богатый набор элементов управления:** Стандартные компоненты, такие как DataGridView для таблиц, DateTimePicker для календаря и NumericUpDown для числовых значений, идеально подходят для ввода параметров спортивных упражнений.
3. **низкий порог вхождения:** В отличие от более сложных технологий (например, WPF), WinForms обладает простой логикой отрисовки и понятной структурой кода.

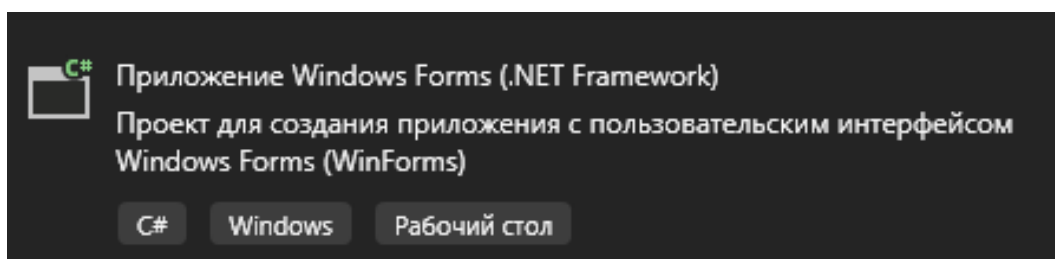


Рисунок 1.6 - Windows Forms

1.5.3 Формат хранения данных JSON

JSON (рис. 1.7) — это легковесный формат обмена данными, основанный на текстовом представлении объектов в виде пар "ключ-значение". Несмотря

на название, он является языконезависимым и поддерживается практически всеми современными языками программирования.

Почему JSON выбран для хранения тренировок:

1. Человечитаемость: Файл с данными можно открыть в обычном блокноте и легко понять его содержимое.
2. простота сериализации: С помощью библиотеки System.Text.Json объекты класса Workout мгновенно преобразуются в строку JSON и сохраняются в файл одной командой.
3. гибкость: В будущем в структуру тренировки можно легко добавить новые поля (например, "Вес" или "Время отдыха"), не нарушая совместимости с уже сохраненными файлами.
4. экономичность: JSON занимает значительно меньше места по сравнению с XML за счет отсутствия громоздких закрывающих тегов.



Рисунок 1.7 - Json

1.5.4 Среда разработки Visual Studio

Microsoft Visual Studio (рис. 1.8) — это полнофункциональная интегрированная среда разработки (IDE), признанная стандартом в индустрии для создания приложений на платформе .NET. Она объединяет в себе

текстовый редактор, средства проектирования интерфейса, компилятор и мощный отладчик.

Функциональные возможности IDE:

1. **IntelliSense:** Система интеллектуального автодополнения кода, которая подсказывает методы и свойства объектов, помогая избегать синтаксических ошибок.
2. **средства отладки (Debugging):** Позволяют останавливать выполнение программы в любой момент (точки останова), проверять значения переменных и отслеживать логику выполнения шаг за шагом.
3. **управление проектом:** Удобный обозреватель решений позволяет структурировать файлы кода, ресурсы (иконки, изображения) и подключаемые библиотеки.
4. **NuGet Package Manager:** Встроенный менеджер пакетов, через который можно легко добавить в проект сторонние библиотеки (например, для создания графиков прогресса тренировок).



Рисунок 1.8 - Visual Studio

2 Специальный раздел

2.1 Подходы к проектированию программного продукта

Выбранная объектно-ориентированная парадигма легла в основу построения модульной структуры приложения. В отличие от процедурного программирования, использование ООП позволило создать гибкую архитектуру, где каждая логическая единица программы обладает четко очерченной ответственностью. Это критически важно для обеспечения надежности системы при манипуляциях с пользовательскими данными.

Особое внимание при проектировании было уделено организации жизненного цикла данных. Инкапсуляция свойств в рамках модели тренировки позволила унифицировать процесс передачи информации между интерфейсным слоем и уровнем хранения. Таким образом, графическая оболочка взаимодействует не с разрозненными переменными, а с целостными объектами, что исключает потерю контекста при сериализации данных в формат JSON.

Реализация событийно-ориентированного подхода позволила добиться высокой интерактивности приложения. В архитектуре WinForms выполнение бизнес-логики жестко привязано к очередям сообщений операционной системы. Это означает, что вычислительные ресурсы задействуются только в моменты непосредственной активности пользователя, что минимизирует нагрузку на систему и обеспечивает мгновенный отклик интерфейса на ввод данных.

Завершающим этапом проектирования стало выстраивание иерархии взаимодействия между визуальными компонентами и внутренними коллекциями. Применение динамических списков типа `List<T>` совместно с событийной моделью обеспечило синхронность отображения: любое изменение во внутреннем массиве данных немедленно транслируется в

					УП.340000.000	Лист.
Изм.	Лист	№ докум.	Подп.	Дата		20

визуальный компонент `ListBox`, поддерживая актуальность информации без необходимости полной перезагрузки интерфейса.

2.2 Проектирование модели данных

Основным элементом системы является информационная сущность «Тренировка». Для её представления в программном коде спроектирован класс `Workout` (Листинг 2).

Листинг 2 - `Workout`

```
public string ExerciseName { get; set; }
public int Sets { get; set; }
public int Reps { get; set; }
public DateTime Date { get; set; }
public override string ToString()
{
    return $"{Date.ToShortDateString()} | {ExerciseName}: {Sets} подходов по {Reps}
повторений";
}
```

В таблице (таб. 2.2) ниже приведено описание структуры данных этой сущности:

Таблица 2.2

Свойство (Поле)	Тип данных (C#)	Описание
ExerciseName	string	Название выполненного упражнения
Sets	int	Количество подходов
Reps	int	Количество повторений в одном подходе
Date	Date	Дата и время проведения занятия

Данная структура является избыточной и достаточной для реализации базового функционала дневника тренировок. Использование типа DateTime позволяет в будущем внедрить функции фильтрации записей по неделям, месяцам или годам.

2.3 Структура и содержание хранилища данных

Для обеспечения сохранности информации между сеансами работы приложения используется локальное хранилище в формате JSON (листинг 2.1). Выбор этого формата обусловлен его иерархической структурой, которая идеально соответствует коллекциям объектов в C#.

Процесс сериализации и десериализации

Взаимодействие с хранилищем происходит в два этапа:

1. Сериализация: Преобразование списка объектов List<Workout> в текстовую строку формата JSON для записи в файл workouts.json.
2. десериализация: Чтение текстового файла и восстановление объектов C# для отображения в интерфейсе программы.

Пример структуры JSON-файла

Содержимое хранилища представляет собой массив объектов. Пример записи в файле workouts.json:

Листинг 2.2 – Использование Json в коде

```
var options = new JsonSerializerOptions { WriteIndented = true };  
string jsonString = JsonSerializer.Serialize(workoutsList, options);  
File.WriteAllText(fileName, jsonString);
```

Листинг 2.3 – пример кода программы

```
[  
    {  
        "ExerciseName": "Приседания со штангой",
```

```

        "Sets": 4,
        "Reps": 12,
        "Date": "2026-05-04T10:30:00"
    },
    {
        "ExerciseName": "Жим гантелей",
        "Sets": 3,
        "Reps": 10,
        "Date": "2026-05-04T11:15:00"
    }
]

```

Организация файловой системы

Файл данных создается в корневой директории приложения. При запуске программа проверяет наличие файла:

1. Если файл существует, данные загружаются в оперативную память.
2. Если файл отсутствует (первый запуск), создается новый пустой список.

Такой подход гарантирует отказоустойчивость приложения и избавляет от необходимости настройки сторонних серверов баз данных.

2.4 Логическая схема работы приложения

Логика приложения строится на циклическом процессе (рис. 2):

1. Сбор данных из элементов интерфейса.
2. валидация (проверка, что название упражнения не пустое, а числа положительные).
3. обновление коллекции (добавление нового экземпляра класса в список).
4. синхронизация с хранилищем (автоматическое или ручное сохранение в JSON).

					УП.340000.000	Лист.
						23
Изм.	Лист	№ докум.	Подп.	Дата		

5. отрисовка (обновление элемента ListBox или DataGridView для визуального подтверждения).



Рисунок 2 – схема работы приложения

3 Технологический раздел

3.1 Разработка пользовательского интерфейса

Интерфейс приложения разработан с использованием графической библиотеки Windows Forms. Основной акцент при проектировании был сделан на лаконичность и удобство ввода данных, чтобы пользователь мог максимально быстро зафиксировать результаты тренировки.

Основные элементы управления:

- Панель ввода данных: Группа элементов, включающая TextBox (для ввода названия упражнения), NumericUpDown (для выбора количества подходов и повторений) и DateTimePicker (для фиксации даты). Использование NumericUpDown вместо обычного текстового поля исключает ввод некорректных данных (букв вместо цифр). кнопки управления (рисунок 3) btnAdd: инициирует сохранение текущего упражнения в список (листинг 3) btnDelete: инициирует удаления выбранного упражнения (листинг 3.1) Центральное место занимает ListBox или DataGridView, в котором отображаются все добавленные записи в хронологическом порядке.

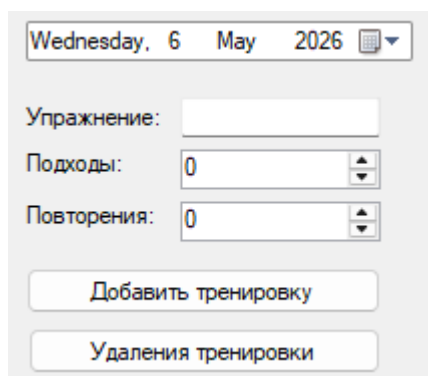


Рисунок 3 – кнопки управления

Листинг 3 – btnAdd

```
if (string.IsNullOrEmpty(txtExercise.Text))  
{
```

					УП.340000.000	Лист.
Изм.	Лист	№ докум.	Подп.	Дата		25

```

        MessageBox.Show("Введите название упражнения!", "Внимание",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }

    Workout newWorkout = new Workout
    {
        ExerciseName = txtExercise.Text,
        Sets = (int)numSets.Value,
        Reps = (int)numReps.Value,
        Date = dtpDate.Value
    };

    workoutsList.Add(newWorkout);
    UpdateWorkoutList();

    SaveData();

    txtExercise.Clear();
    txtExercise.Focus();

```

Листинг 3.1 – btnDelete

```

if (listBoxWorkouts.SelectedIndex != -1)
{
    workoutsList.RemoveAt(listBoxWorkouts.SelectedIndex);
    UpdateWorkoutList();
    SaveData();
}

```

					УП.340000.000	Лист.
						26
Изм.	Лист	№ докум.	Подп.	Дата		

3.2 Программная реализация модели данных

Для представления данных в программе реализован класс Workout. Он инкапсулирует в себе все свойства упражнения. Для корректного отображения объекта в текстовом списке был переопределен метод ToString().\

Листинг 3.2

```
public class Workout
{
    public string ExerciseName { get; set; } // Название
    public int Sets { get; set; }           // Подходы
    public int Reps { get; set; }           // Повторения
    public DateTime Date { get; set; }      // Дата

    // Форматирование вывода для ListBox
    public override string ToString()
    {
        return $"{Date.ToShortDateString()} | {ExerciseName}
- {Sets}x{Reps}";
    }
}
```

3.3 Реализация логики хранения (JSON)

Ключевой особенностью технологического решения является использование пространства имен System.Text.Json. Процесс сохранения реализован как преобразование коллекции List<Workout> в текстовый файл.

Метод сохранения данных:

Листинг 3.3

```
private void SaveWorkouts()
{
```

					УП.340000.000	Лист.
Изм.	Лист	№ докум.	Подп.	Дата		27


```

try
{
    // Настройка красивого вывода JSON (с отступами)
    var options = new JsonSerializerOptions {
WriteIndented = true };

    string jsonString =
JsonSerializer.Serialize(workoutsList, options);
    File.WriteAllText("data.json", jsonString);
}
catch (Exception ex)
{
    MessageBox.Show($"Ошибка при сохранении:
{ex.Message}");
}
}

```

Данный подход обеспечивает высокую скорость работы и малый размер итогового файла данных.

3.4 Обработка событий и валидация

В рамках реализации пользовательского интерфейса особое внимание уделено обеспечению отказоустойчивости системы через внедрение механизмов превентивной валидации. Валидация входных данных в приложении «Планировщик тренировок» рассматривается не только как средство предотвращения ошибок выполнения (Runtime Errors), но и как инструмент обеспечения смысловой целостности локального хранилища JSON.

Применение событийно-ориентированного подхода позволяет изолировать логику проверки данных внутри обработчиков сигналов графических компонентов. Использование метода `string.IsNullOrEmpty()` для анализа текстового ввода обеспечивает более строгий контроль, чем простая проверка на равенство пустой строке, исключая возможность

					УП.340000.000	Лист.
						28
Изм.	Лист	№ докум.	Подп.	Дата		

сохранения записей, состоящих исключительно из пробельных символов. Это гарантирует, что в информационной системе не будут накапливаться некорректные или «пустые» записи, затрудняющие последующий анализ тренировочного процесса.

Процесс обновления визуального представления данных отделен от логики их обработки, что соответствует принципам модульного проектирования. Метод синхронизации коллекции со списком ListBox выполняет роль интерфейсного посредника, гарантируя актуальность отображаемой информации. Такой подход позволяет избежать типичных ошибок рассинхронизации, когда данные в оперативной памяти устройства не соответствуют тому, что видит пользователь на экране.

Дополнительным уровнем защиты служит использование специализированных компонентов управления, таких как NumericUpDown для ввода числовых параметров (подходы, повторения). Это на программном уровне ограничивает диапазон допустимых значений, делая невозможным ввод некорректных символов или отрицательных чисел, что избавляет от необходимости написания громоздких блоков обработки исключений try-catch для каждого поля ввода. Таким образом, архитектура интерфейса сама по себе выступает в роли первичного фильтра валидации, обеспечивая плавное и предсказуемое взаимодействие пользователя с программным продуктом.

3.5 Тестирование работоспособности

Этап тестирования является ключевым звеном технологического цикла разработки, позволяющим подтвердить корректность работы всех модулей приложения в различных сценариях эксплуатации. Основной целью проведенных испытаний стала проверка стабильности взаимодействия между графическим интерфейсом и уровнем персистентности данных, реализованным на базе формата JSON. Особое внимание уделялось поведению программы в

					УП.340000.000	Лист.
Изм.	Лист	№ докум.	Подп.	Дата		29

пограничных состояниях, что критически важно для обеспечения положительного пользовательского опыта и сохранности персональной информации о тренировочном процессе.

В процессе верификации функциональных возможностей была подтверждена высокая пропускная способность алгоритмов обработки списков. Стабильное отображение множественных записей без деградации производительности интерфейса свидетельствует о корректном управлении ресурсами памяти и эффективном использовании динамических коллекций. Проверка механизмов записи и считывания информации подтвердила полную совместимость спроектированной модели данных с методами сериализации, что гарантирует неизменность структуры объектов при их долгосрочном хранении на физическом носителе.

Анализ отказоустойчивости показал, что архитектура приложения обладает встроенными средствами обработки исключительных ситуаций, связанных с внешними факторами, такими как отсутствие или повреждение файлов конфигурации. Реализованная логика автоматической инициализации структуры данных при первом запуске или в случае критических ошибок файловой системы позволяет программе сохранять работоспособность без вмешательства разработчика. Это делает программный продукт автономным и пригодным для использования пользователями с любым уровнем технической подготовки.

Завершающим этапом тестирования стала проверка целостности визуальной составляющей при различных манипуляциях с данными. Отсутствие графических артефактов и корректная работа триггеров обновления экрана подтверждают правильность выбранной событийно-ориентированной модели. Таким образом, результаты комплексных испытаний позволяют сделать вывод о полной готовности «Планировщика тренировок» к эксплуатации и его соответствии всем заявленным в техническом задании критериям качества.

					УП.340000.000	Лист.
Изм.	Лист	№ докум.	Подп.	Дата		30

Заключение

В ходе выполнения курсового проекта была проведена работа по проектированию и разработке программного продукта «Планировщик спортивных тренировок» на языке программирования C#. На основании проведенного исследования и процесса разработки можно сделать следующие выводы:

1. Анализ предметной области показал высокую востребованность инструментов для индивидуального планирования нагрузок. Было выявлено, что для персонального использования наиболее оптимальным является создание легкого настольного приложения, не требующего постоянного подключения к сети Интернет.

2. обоснован выбор технологий: использование языка C# и платформы Windows Forms позволило в краткие сроки создать функциональный графический интерфейс. Выбор формата JSON для хранения данных обеспечил высокую скорость работы с файловой системой и человекочитаемость базы данных.

3. проектирование и реализация: в соответствии с принципами объектно-ориентированного программирования была разработана архитектура приложения, центральным звеном которой стал класс Workout. Реализованный интерфейс обеспечивает интуитивно понятный ввод данных, сводя к минимуму риск ошибок со стороны пользователя за счет использования специализированных элементов управления.

4. функциональные возможности: разработанное программное обеспечение позволяет пользователю формировать список упражнений, указывать параметры нагрузки (подходы, повторения), выбирать дату занятия и сохранять накопленную историю в локальный файл. Функции автоматической загрузки данных при старте программы обеспечивают непрерывность тренировочного процесса.

					УП.340000.000	Лист.
Изм.	Лист	№ докум.	Подп.	Дата		31

5. тестирование: проведенные проверки подтвердили стабильность работы приложения, корректность обработки исключительных ситуаций (например, попытка ввода пустых значений) и надежность сохранения информации.

Цель работы, заключающаяся в создании функционального приложения для планирования и учета тренировок, **достигнута**. Все поставленные задачи выполнены в полном объеме.

Практическая значимость проекта заключается в возможности использования разработанной программы как готового инструмента для спортсменов-любителей, так и в качестве базы для дальнейшего расширения функционала (добавление графиков прогресса, интеграция справочников упражнений и т.д.).

					УП.340000.000	Лист.
						32
Изм.	Лист	№ докум.	Подп.	Дата		

Список использованных источников

1. Вильямс, А. Программирование на C# 12 и .NET 8. Полное руководство / А. Вильямс. — М.: Диалектика, 2024. — 1024 с.
2. Троелсен, Э. Язык программирования C# 10 и платформа .NET 6 / Э. Троелсен, Ф. Джепикс. — СПб.: Питер, 2023. — 1216 с.
3. Рихтер, Д. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C# / Д. Рихтер. — СПб.: Питер, 2019. — 896 с.
4. Документация Microsoft Learn. Общие сведения о Windows Forms [Электронный ресурс].
5. Документация Microsoft Learn. Сериализация и десериализация JSON в .NET [Электронный ресурс].
6. ГОСТ 7.1-2003. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — М.: Изд-во стандартов, 2004. — 170 с.
7. □ Купер, А. Об интерфейсе. Основы проектирования взаимодействий / А. Купер, Р. Рейман, Д. Кронин, К. Носсел. — СПб.: Питер, 2020. — 720 с.
8. □ Нильсен, Я. Дизайн навигации. Веб-дизайн / Я. Нильсен, Х. Лоранжер. — М.: Вильямс, 2018. — 448 с.
9. □ Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. — СПб.: Питер, 2021. — 352 с.
10. □ Фаулер, М. Шаблоны корпоративных приложений / М. Фаулер. — М.: Вильямс, 2020. — 544 с.
11. □ Кнут, Д. Э. Искусство программирования. Том 1. Основные алгоритмы / Д. Э. Кнут. — М.: Вильямс, 2019. — 720 с.
12. □ Кормен, Т. Алгоритмы: вводный курс / Т. Кормен. — М.: Вильямс, 2021. — 208 с.

					УП.340000.000	Лист.
Изм.	Лист	№ докум.	Подп.	Дата		33

13. □ ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения. — М.: Стандартинформ, 2010.

14. □ ГОСТ 2.105-95. Единая система конструкторской документации. Общие требования к текстовым документам. — М.: Стандартинформ, 2005.

					УП.340000.000	Лист.
						34
Изм.	Лист	№ докум.	Подп.	Дата		